

Universidade Federal do Rio Grande do Norte
Estruturas Algbricas DIM0167 - 2026.2

Especificação Algébrica de Sistema de Biblioteca em CafeOBJ

Amauri Lacerda de Brito Junior
Kael Amir Pina Paraguassu Silva
Gabriel Victor da Silva
Pedro Lucas Medeiros do Nascimento
Samuel Morais da Rocha e Silva

Natal, 2 de novembro de 2026.

1 Introdução

Este trabalho apresenta uma especificação algébrica de um sistema de biblioteca em CafeOBJ. O modelo descreve livros, usuários e empréstimos por meio de tipos, operadores e equações, com foco nas regras de negócio e na consistência do estado. A validação experimental utiliza redução de termos e testes de operações válidas e inválidas.

2 Visão Geral do Sistema e Regras de Negócio

O sistema de biblioteca é composto por três coleções principais:

- Acervo: multiconjunto de livros pertencentes à biblioteca.
- Usuários: multiconjunto de usuários registrados.
- Empréstimos: multiconjunto de pares (livro, usuário) que representam empréstimos ativos.

Cada livro possui um identificador único (LivroId), uma obra associada (ObraId), título, autor e ano. Cada usuário possui identificador único (UsuarioId), nome e e-mail. As regras de negócio definidas são:

1. Um livro só pode ser emprestado se pertencer ao acervo.
2. Um livro não pode ser emprestado se já estiver emprestado.
3. A devolução de um livro deve torná-lo novamente disponível.
4. Um usuário só pode devolver um livro que esteja emprestado a ele.
5. A remoção de um livro só é permitida quando ele estiver disponível.
6. As identificações de livros e usuários devem ser únicas em suas respectivas coleções.
7. Em um estado válido, a quantidade de livros disponíveis mais a quantidade de livros emprestados deve ser igual ao total do acervo.
8. Um empréstimo exige que o usuário esteja registrado.
9. Um usuário só pode ser removido se não possuir empréstimos ativos.
10. O cadastro exige livro com título e autor não vazios e ano positivo, ou usuário com nome e e-mail não vazios, além de ID ainda não cadastrado.

As transições globais aplicam essas condições por meio de guardas; quando uma condição falha, preservam o estado recebido. Os predicados `valido?` e `emprestimosValidos?` permitem consultar a consistência do estado. Um usuário pode possuir vários exemplares simultaneamente. `LivroId` identifica o exemplar físico, enquanto `ObraId` pode ser compartilhado por exemplares da mesma obra. O catálogo oferece busca por ID, título e autor, além da contagem de exemplares por obra.

3 Assinatura

3.1 *Sorts* e *Subsorts*

A especificação declara os seguintes *sorts* e *subsorts*:

- `LivroId`, `UsuarioId`, `ObraId`: identificadores atômicos.
- `Livro`, `Usuario`: entidades.
- `Acervo`, `ConjuntoUsuarios`, `ConjuntoEmprestimos`: coleções monoidais.
- `Emprestimo`: relação livro–usuário.
- `Biblioteca`: estado global visível.
- `LivroOuErro`: *supersort* que inclui Livro e os construtores de erro `livroInexistente` e `livroDuplicado`.
- Subsorts: Livro < LivroOuErro, Livro < Acervo, Usuario < ConjuntoUsuarios, Emprestimo < ConjuntoEmprestimos.

3.2 Operadores e Construtores

As tabelas abaixo agrupam a assinatura por módulo. C indica construtor (`constr`); D indica operador definido por equações. A sequência à esquerda de $\to$ informa os tipos dos argumentos e sua quantidade; uma sequência vazia indica aridade zero. Predicados retornam `Bool`. Para compactar as tabelas, utilizam-se as abreviações de tipos: $LI =$ `LivroId`, $UI =$ `UsuarioId`, $OI =$ `ObraId`, $L =$ `Livro`, $U =$ `Usuario`, $A =$ `Acervo`, $US =$ `ConjuntoUsuarios`, $ES =$ `ConjuntoEmprestimos`, $B =$ `Biblioteca`, $R =$ `LivroOuErro`, $S =$ `String`, $N =$ `Nat` e $Q =$ `Bool`. Os tipos `String`, `Nat` e `Bool` são importados da biblioteca padrão. `IDENTIFICADORES` declara apenas os três tipos de IDs; as constantes concretas são introduzidas nos exemplos.

Operador (módulos LIVRO e USUARIO)	Assinatura	Tipo
`livro`	$LI\ OI\ S\ S\ N \to L$	C
`usuario`	$UI\ S\ S \to U$	C
`id`	$L \to LI$; $U \to UI$	D
`obra`; `ano`	$L \to OI$; $L \to N$	D
`titulo`, `autor`	$L \to S$	D
`nome`, `email`	$U \to S$	D
`valido?`	$L \to Q$; $U \to Q$	D

Operador (módulo ACERVO)	Assinatura	Tipo
`vazioAcervo`; `__`	$\rightarrow A$; $A\ A \rightarrow A$	C
`livroInexistente`, `livroDuplicado`	$\rightarrow R$	C
`iniciarAcervo`	$\rightarrow A$	D
`adicionarLivro`	$L\ A \rightarrow A$	D
`removerDoAcervo`, `filtrarId`	$LI\ A \rightarrow A$	D
`buscarLivro`; `resultadoBusca`	$LI\ A \rightarrow R$; $A \rightarrow R$	D
`buscarTitulo`, `tituloDeAutor`	$S\ A \rightarrow A$	D
`qtdAcervo`; `qtdExemplaresObra`	$A \rightarrow N$; $OI\ A \rightarrow N$	D
`pertenceAcervo?`	$LI\ A \rightarrow Q$	D
`idsUnicos?`, `todosValidos?`	$A \rightarrow Q$	D

Operador (módulo COLECAO-USUARIOS)	Assinatura	Tipo
`vazioUsuarios`; `__`	$\rightarrow US$; $US\ US \rightarrow US$	C
`iniciarColecao`	$\rightarrow US$	D
`adicionarUsuario`	$U\ US \rightarrow US$	D
`removerUsuario`	$UI\ US \rightarrow US$	D
`qtdUsuarios`	$US \rightarrow N$	D
`pertence?`	$UI\ US \rightarrow Q$	D
`idsUnicos?`, `todosValidos?`	$US \rightarrow Q$	D
Operador (módulo EMPRESTIMO)	Assinatura	Tipo
`semEmprestimo`; `__`	$\rightarrow ES$; $ES\ ES \rightarrow ES$	C
`emprestar` (registro de empréstimo)	$LI\ UI \rightarrow ES$	C
`iniciarEmprestimos`	$\rightarrow ES$	D
`emprestar` (inserção na coleção)	$LI\ UI\ ES \rightarrow ES$	D
`devolver`	$LI\ ES \rightarrow ES$	D
`qtdEmprestimos`	$ES \rightarrow N$	D
`emprestado?`	$LI\ ES \rightarrow Q$	D
`temLivro?`	$UI\ LI\ ES \rightarrow Q$; $UI\ ES \rightarrow Q$	D
`livrosUnicos?`	$ES \rightarrow Q$	D

Operador (módulo BIBLIOTECA)	Assinatura	Tipo
`biblioteca`; `bibliotecaVazia`	$A\ US\ ES \rightarrow B$; $\rightarrow B$	C; D
`acervo`; `usuarios`; `emprestimos`	$B \rightarrow A$; $B \rightarrow US$; $B \rightarrow ES$	D
`registrarLivro`; `removerLivro`	$L\ B \rightarrow B$; $LI\ B \rightarrow B$	D
`registrarUsuario`; `removerUsuario`	$U\ B \rightarrow B$; $UI\ B \rightarrow B$	D
`emprestarLivro`, `devolverLivro`	$LI\ UI\ B \rightarrow B$	D
`qtdAcervo`, `qtdUsuarios`, `qtdEmprestimos`, `qtdDisponiveis`	$B \rightarrow N$	D
`buscarLivro`	$LI\ B \rightarrow R$	D
`buscarTitulo`, `tituloDeAutor`	$S\ B \rightarrow A$	D
`qtdExemplaresObra`	$OI\ B \rightarrow N$	D
`contarDisponiveis`	$A\ ES \rightarrow N$	D
`pertenceAcervo?`, `emprestado?`, `disponivel?`	$LI\ B \rightarrow Q$	D
`usuarioRegistrado?`	$UI\ B \rightarrow Q$	D
`temLivro?`	$UI\ LI\ B \rightarrow Q$; $UI\ B \rightarrow Q$	D
`emprestimosValidos?`, `valido?`	$B \rightarrow Q$	D

3.3 Variáveis

As equações utilizam variáveis tipadas, quantificadas universalmente em seus respectivos módulos. A tabela reúne as variáveis empregadas neste relatório; alguns nomes locais foram uniformizados para evitar ambiguidades entre módulos. As abreviações de tipos das tabelas anteriores não constituem declarações de variáveis em CafeOBJ.

Declaração CafeOBJ	Papel
`vars L L' : Livro .`	Exemplares
`var U : Usuario .`	Usuário
`vars LID LID2 : LivroId .`	Identificadores de exemplares
`var UID : UsuarioId .`	Identificador de usuário
`var OID : ObraId .`	Identificador de obra
`vars T AUT NOM EMAIL : String .`	Atributos textuais
`var ANO : Nat .`	Ano de publicação
`var A : Acervo .`	Coleção de livros
`var US : ConjuntoUsuarios .`	Coleção de usuários
`var ES : ConjuntoEmprestimos .`	Coleção de empréstimos
`var B : Biblioteca .`	Estado global

4 Estruturas de Dados

Os módulos de entidades utilizam `IDENTIFICADORES`; `ACERVO` utiliza `LIVRO`, `COLECAO-USUARIOS` utiliza `USUARIO`, e `EMPRESTIMO` relaciona IDs de exemplares e usuários. O módulo `BIBLIOTECA` integra as três coleções em um estado visível.

Cada coleção é um monóide comutativo. O operador binário `__` representa composição por justaposição: `L A`, por exemplo, reúne um livro e um acervo. Seus atributos `assoc`, `comm` e `id:` expressam associatividade, comutatividade e elemento neutro. Os neutros são `vazioAcervo`, `vazioUsuarios` e `semEmprestimo`. Assim, agrupamento e ordem não alteram a coleção; repetições são conservadas e contadas. As inclusões por subsorts permitem usar um `Livro` ou um `Usuario` como coleção unitária. O construtor de empréstimo produz diretamente um `ConjuntoEmprestimos`.

O estado global é a tripla `biblioteca(A, US, ES)`. Suas projeções recuperam os componentes sem modificá-los:

```
eq bibliotecaVazia = biblioteca(vazioAcervo, vazioUsuarios,
   semEmprestimo) .
eq acervo(biblioteca(A, US, ES)) = A .
eq usuarios(biblioteca(A, US, ES)) = US .
eq emprestimos(biblioteca(A, US, ES)) = ES .
```

5 Equações

Em CafeOBJ, `eq` declara uma igualdade usada na redução de termos; `if ... then ... else ... fi` seleciona o resultado conforme uma condição booleana. As projeções de entidades recuperam os campos de seus construtores, como `id(livro(LID, OID, T, AUT, ANO)) = LID`. As validações locais são:

```
eq valido?(livro(LID, OID, T, AUT, ANO)) =
  (T =/= "") and (AUT =/= "") and (ANO > 0) .
```

```
eq valido?(usuario(UID, NOM, EMAIL)) =
  (NOM =/= "") and (EMAIL =/= "") .
```

5.1 Acervo

As equações abaixo representam inserção guardada, remoção de todas as ocorrências de um ID e contagem recursiva, incluindo o caso vazio:

```
eq adicionarLivro(L, A) =
  if not(pertenceAcervo?(id(L), A)) and valido?(L)
  then L A else A fi .
eq removerDoAcervo(LID, vazioAcervo) = vazioAcervo .
eq removerDoAcervo(LID, L A) =
  if LID == id(L) then removerDoAcervo(LID, A)
  else L removerDoAcervo(LID, A) fi .
eq qtdAcervo(vazioAcervo) = 0 .
eq qtdAcervo(L A) = 1 + qtdAcervo(A) .
```

A busca filtra o ID e distingue ausência, resultado único e duplicidade:

```
eq buscarLivro(LID, A) = resultadoBusca(filtrarId(LID, A)) .
eq filtrarId(LID, vazioAcervo) = vazioAcervo .
eq filtrarId(LID, L A) =
  if LID == id(L) then L filtrarId(LID, A)
  else filtrarId(LID, A) fi .
eq resultadoBusca(vazioAcervo) = livroInexistente .
eq resultadoBusca(L) = L .
eq resultadoBusca(L L' A) = livroDuplicado .
```

`buscarTitulo` e `tituloDeAutor` também filtram o acervo, por igualdade de título ou autor; devolvem `vazioAcervo` quando não há correspondência. `qtdExemplaresObra` conta os livros com o `ObraId` informado. A coleção de usuários segue a mesma recursão de inserção, remoção e contagem, com `vazioUsuarios` como caso base e `pertence?` para verificar o ID; a inserção também exige `valido?(U)`. A remoção de um ID ausente preserva a coleção.

5.2 Empréstimos e devoluções

Na coleção de empréstimos, a inserção evita repetir o mesmo par livro–usuário; a devolução remove os registros do exemplar. As restrições relativas ao acervo e ao cadastro são aplicadas pelas transições globais.

```
eq emprestar(LID, UID, ES) =
  if temLivro?(UID, LID, ES) then ES
  else emprestar(LID, UID) ES fi .
eq devolver(LID, semEmprestimo) = semEmprestimo .
eq devolver(LID, emprestar(LID2, UID) ES) =
  if LID == LID2 then devolver(LID, ES)
  else emprestar(LID2, UID) devolver(LID, ES) fi .
eq qtdEmprestimos(semEmprestimo) = 0 .
eq qtdEmprestimos(emprestar(LID, UID) ES) = 1 + qtdEmprestimos(ES) .
```

`emprestado?` procura o ID do livro; `temLivro?` procura um par usuário–livro, ou qualquer empréstimo do usuário na versão de dois argumentos. Esses predicados retornam `false` sobre a coleção vazia.

5.3 Transições globais

As seis transições recebem um estado e produzem outro. Os registros delegam às inserções guardadas das coleções; as remoções exigem livro disponível ou usuário sem empréstimos. Emprestar exige disponibilidade e cadastro, enquanto devolver exige que o livro esteja com o usuário informado.

```
eq registrarLivro(L, biblioteca(A, US, ES)) =
  biblioteca(adicionarLivro(L, A), US, ES) .
eq registrarUsuario(U, biblioteca(A, US, ES)) =
  biblioteca(A, adicionarUsuario(U, US), ES) .
eq removerLivro(LID, biblioteca(A, US, ES)) =
  if disponivel?(LID, biblioteca(A, US, ES))
  then biblioteca(removerDoAcervo(LID, A), US, ES)
  else biblioteca(A, US, ES) fi .
eq removerUsuario(UID, biblioteca(A, US, ES)) =
  if not(temLivro?(UID, ES))
  then biblioteca(A, removerUsuario(UID, US), ES)
  else biblioteca(A, US, ES) fi .
eq emprestarLivro(LID, UID, biblioteca(A, US, ES)) =
  if disponivel?(LID, biblioteca(A, US, ES)) and
     usuarioRegistrado?(UID, biblioteca(A, US, ES))
  then biblioteca(A, US, emprestar(LID, UID, ES))
  else biblioteca(A, US, ES) fi .
eq devolverLivro(LID, UID, biblioteca(A, US, ES)) =
  if temLivro?(UID, LID, biblioteca(A, US, ES))
  then biblioteca(A, US, devolver(LID, ES))
  else biblioteca(A, US, ES) fi .
```

Os observadores globais consultam as coleções pelas projeções. Por exemplo, `usuarioRegistrado?(UID, B)` reduz a `pertence?(UID, usuarios(B))`, e `buscarTitulo(T, B)` reduz a `buscarTitulo(T, acervo(B))`. A disponibilidade e sua contagem são:

```
eq disponivel?(LID, B) =
  pertenceAcervo?(LID, B) and not emprestado?(LID, B) .
eq qtdDisponiveis(B) = contarDisponiveis(acervo(B), emprestimos(B)) .
eq contarDisponiveis(vazioAcervo, ES) = 0 .
eq contarDisponiveis(L A, ES) =
  if emprestado?(id(L), ES) then contarDisponiveis(A, ES)
  else 1 + contarDisponiveis(A, ES) fi .
```

5.4 Invariantes globais

`idsUnicos?` verifica ausência de IDs repetidos no acervo e nos usuários; `todosValidos?` verifica os atributos de cada entidade. `livrosUnicos?` impede que um exemplar apareça em mais de um registro de empréstimo. Esses predicados retornam `true` nas coleções vazias e percorrem as coleções não vazias. A consistência das referências e a validade global são definidas por:

```
eq emprestimosValidos?(biblioteca(A, US, semEmprestimo)) = true .
eq emprestimosValidos?(biblioteca(A, US, emprestar(LID, UID) ES)) =
  pertenceAcervo?(LID, A) and pertence?(UID, US) and
  emprestimosValidos?(biblioteca(A, US, ES)) .
eq valido?(biblioteca(A, US, ES)) =
  idsUnicos?(A) and todosValidos?(A) and
  idsUnicos?(US) and todosValidos?(US) and
  livrosUnicos?(ES) and emprestimosValidos?(biblioteca(A, US, ES)) .
```

Em estados válidos, cada exemplar pertence a exatamente uma das categorias disponível ou emprestado, e cada empréstimo corresponde a um exemplar do acervo. Portanto, `qtdDisponiveis(B) + qtdEmprestimos(B) = qtdAcervo(B)`. Os predicados permitem detectar estados inconsistentes construídos diretamente com `biblioteca`; as transições preservam a validade quando partem de um estado válido.

6 Tratamento de estados inválidos

6.1 Busca por livro inexistente

6.2 Busca por livro com ID duplicado

6.3 Remoção de entidade inexistente

6.4 Adição de elemento inválido ou com ID inexistente

6.5 Devolução de livro não emprestado

6.6 Estados inválidos em nível global

7 Termos de teste

Conforme os requisitos do trabalho, foram desenvolvidos os termos de teste agregados na Tabela 1, cobrindo as operações básicas e compostas, casos inválidos e aplicação das regras de negócio.

Os exemplos usam o contexto abaixo, aberto após carregar `biblioteca.mod`. `bLivre` contém um livro disponível e dois usuários; `bEmprestada` contém o mesmo livro emprestado a Ana. `l2` e `ux` não estão cadastrados. Cada linha da tabela é avaliada independentemente, sem modificar os estados usados pelas demais.

```
open BIBLIOTECA .
ops l1 l2 : -> LivroId .
ops u1 u2 ux : -> UsuarioId .
op o1 : -> ObraId .
op livro1 : -> Livro .
ops ana bia : -> Usuario .
ops bLivre bEmprestada : -> Biblioteca .
eq livro1 = livro(l1, o1, "T1", "A1", 2020) .
eq ana = usuario(u1, "Ana", "ana@x") .
eq bia = usuario(u2, "Bia", "bia@x") .
eq bLivre = biblioteca(livro1, ana bia, semEmprestimo) .
eq bEmprestada = emprestarLivro(l1, u1, bLivre) .
```

Para reproduzir cada verificação, usa-se `red termo == resultado .` no contexto acima e, ao final, `close`. No teste 12, o total 1 também corresponde a `qtdAcervo(bEmprestada)`. Nos testes 9 e 11, as guardas recusam, respectivamente, o usuário não cadastrado e a devolução por outro usuário. No teste 10, o estado preservado é o posterior ao empréstimo.

As sequências abaixo resumem a reescrita, com as condições já avaliadas. No primeiro exemplo, a inserção aceita o livro válido e a contagem chega a 1:

```
qtdAcervo(adicionarLivro(livro1, vazioAcervo))
  => qtdAcervo(livro1 vazioAcervo)
```

Tabela 1: Termos de teste e resultados esperados.

#	Termo de teste	Resultado esperado
1	`qtdAcervo(adicionarLivro(livro1, vazioAcervo))`	`1`
2	`pertenceAcervo?(l1, adicionarLivro(livro1, vazioAcervo))`	`true`
3	`buscarLivro(l2, adicionarLivro(livro1, vazioAcervo))`	`livroInexistente`
4	`qtdUsuarios(adicionarUsuario(ana, vazioUsuarios))`	`1`
5	`emprestado?(l1, emprestar(l1, u1))`	`true`
6	`temLivro?(u1, l1, emprestar(l1, u1))`	`true`
7	`disponivel?(l1, emprestarLivro(l1, u1, bLivre))`	`false`
8	`qtdEmprestimos(emprestarLivro(l1, u1, bLivre))`	`1`
9	`emprestarLivro(l1, ux, bLivre)`	`bLivre`
10	`removerLivro(l1, emprestarLivro(l1, u1, bLivre))`	`bEmprestada`
11	`devolverLivro(l1, u2, bEmprestada)`	`bEmprestada`
12	`qtdDisponiveis(bEmprestada) + qtdEmprestimos(bEmprestada)`	`1`

```
  => 1 + qtdAcervo(vazioAcervo)
  => 1 + 0 => 1
```

No segundo, o livro emprestado não pode ser removido:

```
removerLivro(l1, emprestarLivro(l1, u1, bLivre))
  => removerLivro(l1, biblioteca(livro1, ana bia, emprestar(l1, u1)))
  => biblioteca(livro1, ana bia, emprestar(l1, u1))
```

A condição `disponivel?` é falsa nesse estado; o resultado é a forma expandida de `bEmprestada`. O símbolo `=>` apenas indica passos de reescrita nesta exposição, não um comando de CafeOBJ.

A suíte `spec/testes.cafe`, executada por `python3 scripts/testar.py -homebrew` no ambiente utilizado, apresentou `165/165` asserções verdadeiras, com 165 reduções. Dessas, 64 pertencem ao arquivo `teste-transicoes-biblioteca.cafe`, que cobre transições, consultas e estados inconsistentes. Esse resultado verifica os casos testados.

O cenário separado `spec/demo.cafe` começa com cinco exemplares e quatro usuários. Após empréstimos, uma tentativa de empréstimo repetido, uma devolução e o cadastro de outro exemplar, `biblioteca1` possui seis livros, três empréstimos e três disponíveis. A demonstração também consulta o catálogo, exercita operações recusadas e verifica `valido?`; ela não integra as 165 asserções da suíte.

8 Análise da especificação

8.1 Justificativa dos *sorts* e operadores

Os *sorts* `LivroId, UsuarioId e ObraId` foram separados para garantir unicidade e evitar confusão entre identificadores de naturezas distintas. `Livro` e `Usuario` são entidades com construtores e projeções, permitindo validação (`valido?`) e consultas. As coleções foram modeladas como monóides comutativos, já que a ordem dos elementos não

é relevante para as consultas, e a composição é naturalmente associativa e comutativa. O *sort* `Biblioteca` consolida o estado global e permite transições atômicas e guardadas.

8.2 Representatividade do modelo

A especificação representa as principais características do sistema: acervo, usuários, empréstimos, disponibilidade, pertinência, contagens e regras de negócio. As transições globais refletem o ciclo de vida de um livro (registro, empréstimo, devolução, remoção) e de um usuário (registro, remoção condicionada). As invariantes `valido?` e `emprestimosValidos?` capturam a consistência global.

8.3 Características não representadas

Algumas características do mundo real não foram modeladas, por não serem exigidas pelo escopo mínimo:

1. datas de empréstimo e devolução, prazos e multas;
2. limites de quantidade de empréstimos por usuário;
3. reservas e filas de espera;
4. histórico de empréstimos encerrados;
5. informações bibliográficas mais específicas (editora, edição etc).

Essas extensões poderiam ser incorporadas com novos *sorts* e equações, sem alterar a estrutura das coleções.

8.4 Propriedades matemáticas expressas

A especificação expressa:

- Monóides comutativos para acervo, usuários e empréstimos, com identidade e associatividade;
- Idempotência de filtros e remoções quando o elemento não existe;
- Conservação de cardinalidade: `qtdDisponiveis(B) + qtdEmprestimos(B) = qtdAcervo(B);`
- Unicidade de identificadores via `idsUnicos?`;
- Validade global via `valido?`.

8.5 Representação e mitigação de operações inválidas

As operações inválidas são tratadas de maneiras distintas nas diferentes partes do sistema:

- Consultas ao acervo: No módulo `acervo.mod`, é utilizado o *supersort* `LivroOuErro` que inclui dois construtores de erro: `livroInexistente`, retornado pela busca a um livro cujo ID não existe no acervo; e `livroDuplicado`), se o ID consultado ocorre múltiplas vezes; Essa decisão permite distinguir a ausência, presença ou quebra da unicidade de um livro a partir de um tipo bem definido.
- Guardas nos modificadores de coleção: Os modificadores são guardados por equações que, mediante falha na pré-condição, retornam a coleção original. Desta forma, todas as operações são totais a nível de coleção. Este comportamento foi implementado nas operações.
  - Adição de livros e usuários: Insere apenas se o objeto - livro ou usuário - é válido e se seu ID já não pertence à coleção; se falhar nestas condições, devolve a coleção.
  - Remoção de livros e usuários: Se o ID do objeto não existe, retorna o estado atual da coleção. Essa abordagem torna a operação idempotente.
  - Devolução de livros: preserva a coleção de empréstimos empréstimo não está presente.
- Guardas nas transições globais: No módulo de `Biblioteca`, as transições globais são verificadas por meio de observadores, `disponivel?, usuarioRegistrado?` e `temLivro?`; se não são satisfeitos, a bibioteca original é retornada. Tal como nos modificadores de coleção, esta abordagem faz o sistema total e seguro, pois a partir de um estado válido, nenhuma transação levaria a um inválido.

A suíte de testes verifica que, após qualquer tentativa inválida, o estado permanece inalterado e os invariantes continuam válidos.